

CLASE 5

ALGORITMOS CODICIOSOS

Elegir ahora sin arrepentirse.

Taller de Programación Competitiva | 2026

LAS GEMAS MÁS VALIOSAS

Problema motivador

Tenemos cinco gemas con valores distintos. Debemos elegir exactamente tres y maximizar la suma de sus valores.

¿Cuáles elegirías?



UNA DECISIÓN OBVIA QUE SÍ FUNCIONA

Elegimos primero 7, luego 4 y finalmente 3.

$$7 + 4 + 3 = 14.$$

Regla

En cada paso elegir la gema de mayor valor que todavía no hemos usado.

Resultado

Después de tres pasos obtenemos las tres gemas más valiosas.

Parece evidente. Aun así, necesitaremos demostrar que ninguna otra elección consigue una suma mayor.

AHORA LAS GEMAS TAMBIÉN PESAN

Podemos cargar como máximo $W = 14$ kilos. Cada gema se toma completa o se deja.

Gema	A	B	C	D	E
Valor	2	5	9	5	11
Peso	1	5	10	4	12
Valor por kilo	2.00	1.00	0.90	1.25	0.92

Tres ideas tentadoras

Elegir primero las más valiosas, las más livianas o las de mayor valor por kilo.

LA AVARICIA ROMPE EL SACO

Ninguna de las tres reglas anteriores encuentra la mejor solución.

Regla	Gemas elegidas	Valor
Mayor valor	E, A	13
Menor peso	A, D, B	12
Mejor razón	A, D, B	12

Solución óptima

Las gemas C y D pesan exactamente 14 y tienen valor total

$$9 + 5 = 14.$$

Una regla puede parecer razonable y fallar en un ejemplo muy pequeño.



PARTE 1 • DECISIONES LOCALES

Una elección sencilla necesita condiciones especiales.



TRES PARTES DE UN PROBLEMA DE OPTIMIZACIÓN

Decisiones

Qué gemas elegimos y cuáles dejamos fuera.

Restricciones

Elegir exactamente tres gemas o no superar una capacidad.

Objetivo

Maximizar el valor total entre todas las soluciones que respetan las restricciones.

Una solución es **factible** si cumple las restricciones. Una solución **óptima** tiene el mejor valor posible.

LA RESPUESTA SEGURA SUELE SER DEMASIADO LENTA

Podríamos enumerar todas las soluciones factibles y quedarnos con la mejor.

Búsqueda exhaustiva

Considera todas las combinaciones. En muchos problemas su tiempo es exponencial.

La esperanza

Encontrar una propiedad que permita construir directamente una solución óptima sin revisar todas las alternativas.

Para elegir M de N gemas existen $\binom{N}{M}$ conjuntos posibles.

¿QUÉ SIGNIFICA SER CODICIOSO?

Un algoritmo codicioso construye una solución paso a paso.

1. Define una regla para decidir qué parece mejor ahora.
2. Toma esa decisión de forma definitiva.
3. Continúa con el problema restante.

La dificultad real

Elegir una opción localmente buena no garantiza una solución globalmente óptima. La regla debe demostrarse para ese problema.

A diferencia del backtracking, no exploramos todas las ramas ni deshacemos una elección para probar otra.

UN CONTRAEJEMPLO BASTA

Queremos formar 6 usando la menor cantidad de monedas de valores $\{1, 3, 4\}$.

Regla codiciosa

Tomar siempre la moneda más grande que no supere lo que falta.

$$6 = 4 + 1 + 1.$$

Usa tres monedas.

Una solución mejor

$$6 = 3 + 3.$$

Usa solamente dos monedas.

El contraejemplo demuestra que esa regla no funciona para todas las denominaciones.

ANTES DE CONFIAR EN UNA REGLA

1. Entender exactamente qué decisión toma en cada paso.
2. Intentar la regla en casos pequeños donde podamos conocer el óptimo por fuerza bruta.
3. Intentar romper la regla. Buscar empates, capacidades ajustadas y decisiones que bloqueen el futuro.
4. Si sobrevive, intentar una demostración.

Dos resultados posibles

Un contraejemplo descarta la regla. Una colección de ejemplos correctos solamente aumenta nuestra confianza, pero no reemplaza una prueba.



PARTE 2 • INTERCAMBIOS

Transformar un óptimo sin hacerlo peor.



VOLVAMOS A LAS M GEMAS

La regla ordena los valores de mayor a menor y elige los primeros M .

$$v_1 \geq v_2 \geq \dots \geq v_N.$$

Lo que queremos demostrar

Existe una solución óptima que contiene $v_1, v_2, \dots, v_M$.

No necesitamos que la solución óptima sea única. Si hay valores empatados, pueden existir varias elecciones igualmente buenas.

CAMBIAR UNA ELECCIÓN POR OTRA

Supongamos que una solución óptima elige una gema de valor x , pero deja fuera una de valor $y \geq x$ que pertenece a las M mayores.

Antes

Retiramos la gema de valor x .
La solución conserva exactamente $M - 1$ gemas.

Después

Agregamos la gema de valor y .
Seguimos eligiendo M gemas y la suma no disminuye porque $y \geq x$.

Este reemplazo es un **intercambio**.

EL ARGUMENTO COMPLETO

1. Comenzamos con alguna solución óptima.
2. Si le falta una de las M gemas mayores, la intercambiamos por una gema elegida de valor menor o igual.
3. La solución sigue siendo factible y su valor no disminuye.
4. Repetimos hasta obtener exactamente la elección codiciosa.

Conclusión

Transformamos una solución óptima en la solución codiciosa sin empeorarla. Por lo tanto, la elección codiciosa también es óptima.

ELEGIR LAS M GEMAS MAYORES

```
1 def maximum_sum(values, m):  
2     ordered = sorted(values, reverse=True)  
3     chosen = ordered[:m]  
4     return sum(chosen)
```

sorted crea una lista ordenada.

reverse=True pide orden descendente.

ordered[:m] toma los primeros m elementos.

TIEMPO $\mathcal{O}(N \log N)$

MEMORIA $\mathcal{O}(N)$



PARTE 3 • INTERVALOS

Una regla ganadora.



LA MAYOR CANTIDAD DE ACTIVIDADES

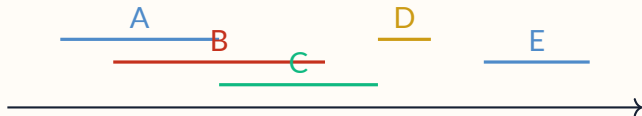
Cada actividad ocupa un intervalo $[s, e)$. Queremos asistir a la mayor cantidad posible sin superponerlas.

Compatibilidad

Si una actividad termina en e , otra puede comenzar exactamente en e . Por eso escribimos el intervalo como $[s, e)$.

Decisión

¿Cuál actividad deberíamos elegir primero?



TRES REGLAS POSIBLES

En cada paso podríamos elegir, entre las actividades compatibles restantes:

Comienza primero

Elegir el menor tiempo de inicio.

Dura menos

Elegir la menor duración.

Termina primero

Elegir el menor tiempo de término.

Las tres parecen ahorrar tiempo. Solamente una funciona siempre.

DOS REGLAS CAEN CON EJEMPLOS PEQUEÑOS

Elegir el inicio más temprano

$[0, 10)$ comienza antes que

$[1, 2), [2, 3), [3, 4), [4, 5)$.

Elegirlo permite una actividad en vez de cuatro.

Elegir la menor duración

$[3, 5)$ es más corto que $[0, 4)$ y $[4, 8)$, pero bloquea ambos. Los otros dos son compatibles entre sí.

No encontramos el criterio correcto preguntando solamente qué actividad usa menos tiempo.

ELEGIR LA QUE TERMINA PRIMERO

Ordenamos por tiempo de término y recorremos en ese orden.

Actividad	Intervalo	Decisión
A	[1, 4)	ELEGIR
B	[2, 6)	DESCARTAR
C	[4, 7)	ELEGIR
D	[7, 8)	ELEGIR
E	[6, 9)	DESCARTAR
F	[9, 11)	ELEGIR

Obtenemos cuatro actividades compatibles.

INTERCAMBIAR LA PRIMERA ACTIVIDAD

Sea G la actividad que termina primero. Consideremos una solución óptima y llamemos O a su primera actividad.

$$\text{fin}(G) \leq \text{fin}(O).$$

El intercambio

Reemplazamos O por G . Toda actividad que comenzaba después del final de O también comienza después del final de G .

Consecuencia

La nueva solución sigue siendo factible y tiene la misma cantidad de actividades. Por lo tanto, existe un óptimo que comienza con G .

Repetimos el argumento con las actividades que empiezan después de G .

SELECCIONAR ACTIVIDADES

```
1 def select_activities(intervals):
2     def finish(interval):
3         return interval[1]
4
5     ordered = sorted(intervals, key=finish)
6     chosen = []
7     last_end = None
8
9     for start, end in ordered:
10         if last_end is None or start >= last_end:
11             chosen.append((start, end))
12             last_end = end
13
14     return chosen
```

key=finish indica que la ordenación debe comparar los tiempos de término.

TIEMPO $\mathcal{O}(N \log N)$

MEMORIA $\mathcal{O}(N)$



PARTE 4 • GREEDY DOBLE

Uno decide y otro comprueba.



PARÉNTESIS BALANCEADOS CON PRESUPUESTO

Para cada prefijo definimos

$$b = \#(- \#).$$

Balanceada

Una secuencia está balanceada si todo prefijo tiene $b \geq 0$ y al final $b = 0$. Por ejemplo, $((()())$ tiene balances

1, 2, 1, 2, 1, 0.

Recibimos una secuencia S de largo par $N \leq 1000$.

- ▶ Invertir el carácter i cuesta a_i .
- ▶ Podemos gastar a lo más B .
- ▶ Declaramos que $($ es lexicográficamente menor que $)$.

Objetivo

Encontrar la secuencia balanceada lexicográficamente menor dentro del presupuesto, o decidir que no existe.

PRIMERA DECISIÓN: INTENTAR «(»

Para

$$S =) () ((, \quad (a_1, \dots, a_6) = (4, 2, 7, 1, 5, 3),$$

el presupuesto cambia cuánto podemos priorizar el orden lexicográfico.

Presupuesto	Respuesta	Costo
7	(()) (	7
13	(()) ()	13
19	((()))	19

Greedy lexicográfico

En cada posición intentamos (. Lo fijamos si el sufijo todavía puede completarse sin exceder el presupuesto. Si no puede, elegimos).

POR QUÉ EL PRIMER GREEDY ES CORRECTO

Supongamos que ya fijamos un prefijo que admite alguna respuesta factible.

1. Intentamos colocar (en la siguiente posición.
2. Si existe una terminación factible, toda respuesta que coloque) en esa posición es lexicográficamente mayor.
3. Elegir (conserva una solución y fija el mejor carácter posible en la primera posición aún no decidida.
4. Si no existe tal terminación, ninguna respuesta válida puede usar (. Debemos colocar).

La pregunta que queda

Después de probar un carácter, calculamos el costo mínimo de completar el sufijo y lo comparamos con el presupuesto restante.

EL SUBPROBLEMA: COSTO MÍNIMO DE UN SUFIJO

Tras probar el siguiente carácter, conocemos el balance b y quedan m posiciones. Si $b > m$, automáticamente no se puede. Supongamos $b \leq m$.

Si el sufijo contiene x aperturas y $m - x$ cierres, su balance final es

$$b + x - (m - x) = 0.$$

Por lo tanto, necesitamos exactamente

$$x = \frac{m - b}{2}$$

aperturas. Como N es par y b viene de un prefijo, $m - b$ también es par.

La reducción

Debemos elegir qué x posiciones serán (, con costo mínimo y sin permitir que algún prefijo tenga balance negativo.

La cantidad de aperturas fija el balance final. La restricción de los prefijos decide dónde deben estar.

PARTIR CON TODOS LOS CIERRES

Suponemos inicialmente que cada posición del sufijo contiene). Al recorrer la posición i hacemos tres cosas.

1. Pagamos $c_i()$ y restamos 1 al balance.
2. Guardamos el costo adicional de convertir esa posición en apertura

$$\Delta_i = c_i(() - c_i()).$$

3. Si el balance llega a -1 , debemos convertir alguna posición ya vista en (. Elegimos el menor Δ_i disponible.

Una reparación

Cambiar) por (suma 2 al balance actual y a todos los balances posteriores.
Un min-heap permite extraer el menor Δ_i .

UNA REPARACIÓN PASO A PASO

El prefijo fijado tiene balance $b = 1$. El sufijo original es $)))$ y sus costos de inversión son 1, 7, 4.

Posición	Balance	Δ disponibles	Acción
1	0	$\{1\}$	Ninguna
2	-1	$\{1, 7\}$	Usar 1. El balance vuelve a 1
3	0	$\{4, 7\}$	

Cambiamos la primera posición del sufijo. Obtenemos $()$), terminamos con balance cero y pagamos 1.

POR QUÉ LA REPARACIÓN MÁS BARATA ES ÓPTIMA

Cuando el balance llega a -1 , una nueva apertura entre las posiciones vistas es obligatoria.

1. Las reparaciones anteriores ya mantienen válidos todos los prefijos previos.
2. Cualquier posición disponible suma los mismos 2 al balance y evita la nueva violación.
3. Elegir el menor Δ_i satisface la obligación con el menor costo adicional posible.

Repetir el argumento

Aplicamos la misma elección cada vez que aparece una violación. La ecuación $x = (m - b)/2$ garantiza que hacemos exactamente las reparaciones necesarias y terminamos con balance cero.

PREPARAR EL COSTO DE CADA ELECCIÓN

Para cada posición guardamos por separado cuánto cuesta escribir cada paréntesis.

```
1 def character_costs(s, price):
2     open_cost = []
3     close_cost = []
4     for i in range(len(s)):
5         if s[i] == "(":
6             open_cost.append(0)
7             close_cost.append(price[i])
8         else:
9             open_cost.append(price[i])
10            close_cost.append(0)
11    return open_cost, close_cost
```

Así, el resto del código trabaja directamente con el costo de elegir (o).

COSTO MÍNIMO PARA COMPLETAR UN SUFIJO

```
1 from heapq import heappop, heappush
2 from math import inf
3
4 def min_suffix_cost(open_cost, close_cost, start, balance):
5     remaining = len(open_cost) - start
6     if balance < 0 or balance > remaining:
7         return inf
8
9     deltas, total = [], 0
10    for i in range(start, len(open_cost)):
11        total += close_cost[i]
12        heappush(deltas, open_cost[i] - close_cost[i])
13        balance -= 1
14        if balance == -1:
15            total += heappop(deltas)
16            balance += 2
17    return total
```

CONSTRUIR LA RESPUESTA LEXICOGRÁFICA

```
1 def smallest_balanced(s, price, budget):
2     open_cost, close_cost = character_costs(s, price)
3     if min_suffix_cost(open_cost, close_cost, 0, 0) > budget:
4         return None
5
6     answer, balance = [], 0
7     for i in range(len(s)):
8         suffix = min_suffix_cost(open_cost, close_cost,
9                                   i + 1, balance + 1)
10        if open_cost[i] + suffix <= budget:
11            answer.append("(")
12            budget -= open_cost[i]
13            balance += 1
14        else:
15            answer.append(")")
16            budget -= close_cost[i]
17            balance -= 1
18    return "".join(answer)
```

DOS GREEDYS CON RESPONSABILIDADES DISTINTAS

Greedy exterior

Intenta (para obtener la respuesta lexicográficamente menor. Solo pregunta si queda alguna terminación factible.

Greedy interior

Calcula el costo mínimo de una terminación. Si un prefijo se vuelve inválido, lo repara usando el menor Δ disponible.

Cada consulta recorre un sufijo con un heap. Hacemos una al inicio y otra en cada posición.

TIEMPO $\mathcal{O}(N^2 \log N)$

MEMORIA $\mathcal{O}(N)$

LO QUE DEBEMOS RECORDAR

Diseñar

Un algoritmo codicioso toma decisiones locales definitivas. La regla es específica para cada problema.

Refutar

Un solo contraejemplo basta para demostrar que una regla no funciona siempre.

Demostrar

Un intercambio transforma una solución óptima para que respete la elección codiciosa sin empeorarla.